

MEMORIA TÉCNICA

Clasificador de Enfermedad Cardíaca mediante Machine Learning

Proyecto 3 - CE en Desarrollo de aplicaciones en lenguaje Python



Índice

1. Descripción del proyecto y su propósito.....	1
2. Tecnologías usadas y por qué se eligieron.....	2
3. Arquitectura del sistema.....	2
Flujo de datos.....	2
Estructura de carpetas.....	2
4. Modelo de datos.....	3
Estructura del dataset.....	3
Transformaciones aplicadas.....	4
5. Funcionalidades principales.....	4
preprocess(path) — preprocessing.py.....	4
train(data_path) — train_model.py.....	4
evaluate(data_path) — evaluation.py.....	4
6. Explicación de las partes más complejas del código.....	4
Imputación agrupada por target.....	4
Prevención de data leakage en el escalado.....	5
Validación cruzada antes del entrenamiento final.....	5
7. Seguridad implementada.....	5
8. Tests y resultados.....	6
Validación cruzada (sobre train).....	6
Métricas sobre test (184 pacientes).....	6
Rendimiento por sexo - Recall.....	6
Errores críticos del modelo seleccionado (Random Forest).....	6
9. Decisiones de diseño justificadas.....	6
Imputación en lugar de eliminación de filas con ceros.....	6
One-Hot Encoding en lugar de Label Encoding.....	6
StandardScaler en lugar de MinMaxScaler.....	7
Priorizar Recall sobre Accuracy en la selección del modelo.....	7
Arquitectura modular en src/.....	7
10. Posibles mejoras futuras.....	7

1. Descripción del proyecto y su propósito

El proyecto consiste en un sistema de clasificación supervisada capaz de determinar si un paciente presenta enfermedad cardíaca a partir de indicadores biomédicos clínicos.

El propósito es demostrar el ciclo completo de un proyecto de Data Science:

- Carga y limpieza de datos clínicos reales
- Análisis exploratorio exhaustivo (EDA)
- Preprocesamiento y transformación de variables
- Entrenamiento y comparación de múltiples modelos de clasificación
- Evaluación orientada al contexto médico, priorizando el Recall sobre la Accuracy

El sistema clasifica cada paciente en una de dos categorías:

- 0 → No presenta enfermedad cardíaca
- 1 → Presenta enfermedad cardíaca

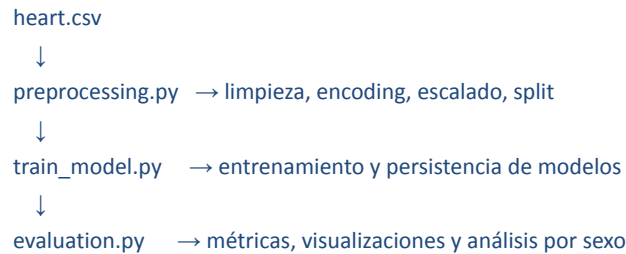
2. Tecnologías usadas y por qué se eligieron

Tecnología	Versión	Justificación
Python	3.x	Lenguaje estándar en Data Science con el ecosistema más maduro
Pandas	3.0.2	Manipulación y análisis de DataFrames de forma eficiente
NumPy	2.4.1	Operaciones numéricas y manejo de arrays
Matplotlib	3.10.8	Visualizaciones base con control total sobre las gráficas
Seaborn	0.13.2	Visualizaciones estadísticas de alto nivel sobre Matplotlib
Scikit-learn	1.8.0	Preprocesamiento, modelos de clasificación y métricas
XGBoost	-	Gradient boosting de alto rendimiento en datos tabulares
Joblib	1.5.3	Serialización y persistencia de modelos entrenados
Jupyter Notebook	1.1.1	Entorno interactivo para el análisis exploratorio

3. Arquitectura del sistema

Flujo de datos

El sistema sigue una arquitectura modular por capas, donde cada módulo tiene una responsabilidad única y bien definida:



Estructura de carpetas

```
CLASIFICADOR-DE-ESTADO-DE-SALUD-MEDIANTE-MACHINE-LEARNING/
├── data/
│   └── heart.csv          # Dataset original (918 registros)
├── models/
│   ├── logistic_regression.pkl
│   ├── random_forest.pkl
│   ├── xgboost.pkl
│   └── scaler.pkl         # StandardScaler entrenado en train
├── notebooks/
│   └── analisis.ipynb     # EDA completo
├── src/
│   ├── preprocessing.py   # Pipeline de preprocesamiento
│   ├── train_model.py     # Entrenamiento y guardado
│   └── evaluation.py      # Evaluación y visualizaciones
├── images/               # Gráficas generadas
├── README.md
└── requirements.txt
```

4. Modelo de datos

Los datos se gestionan como un DataFrame de Pandas cargado desde un archivo CSV con 918 registros y 12 variables.

Estructura del dataset

Variable	Tipo original	Tipo tras preprocesamiento	Descripción
Age	int64	float64 (escalado)	Edad del paciente
Sex	str	uint8 (One-Hot)	Sexo (M/F)
ChestPainType	str	uint8 (One-Hot)	Tipo de dolor torácico
RestingBP	int64	float64 (escalado)	Presión arterial en reposo
Cholesterol	int64	float64 (escalado)	Nivel de colesterol
FastingBS	int64	int64	Glucosa en ayunas > 120 mg/dl
RestingECG	str	uint8 (One-Hot)	Resultado del ECG
MaxHR	int64	float64 (escalado)	Frecuencia cardíaca máxima
ExerciseAngina	str	uint8 (One-Hot)	Angina por ejercicio
Oldpeak	float64	float64 (escalado)	Depresión segmento ST
ST_Slope	str	uint8 (One-Hot)	Pendiente segmento ST
HeartDisease	int64	int64 (target)	Variable objetivo

Transformaciones aplicadas

- Imputación: ceros de Cholesterol y RestingBP reemplazados por la mediana agrupada por HeartDisease
- Agrupación: categoría TA fusionada con NAP en ChestPainType por similitud y bajo número de muestras
- Encoding: One-Hot Encoding en variables categóricas con `pd.get_dummies()`
- Escalado: StandardScaler aplicado sólo sobre train y transformado sobre test para evitar data leakage

5. Funcionalidades principales

preprocess(path) - preprocessing.py

Ejecuta el pipeline completo de preprocesamiento y devuelve los conjuntos de train y test listos para el entrenamiento.

```
x_train, X_test, y_train, y_test, scaler = preprocess('../data/heart.csv')
```

train(data_path) - train_model.py

Entrena los tres modelos con validación cruzada de 5 folds y los guarda en models/.

```
trained, X_test, y_test = train('../data/heart.csv')
```

Output en consola:

```
Entrenando Logistic Regression... CV Accuracy: 0.849 ± 0.026
Entrenando Random Forest...      CV Accuracy: 0.846 ± 0.019
Entrenando XGBoost...            CV Accuracy: 0.849 ± 0.010
```

evaluate(data_path) - evaluation.py

Carga los modelos guardados, calcula métricas sobre test y genera visualizaciones en images/.

```
results = evaluate('../data/heart.csv')
```

6. Explicación de las partes más complejas del código

Imputación agrupada por target

En lugar de imputar con la mediana global, se calcula la mediana por separado para enfermos (HeartDisease=1) y sanos (HeartDisease=0). Esto preserva las diferencias clínicas entre grupos:

```
for col in ['Cholesterol', 'RestingBP']:
    df[col] = df.groupby('HeartDisease')[col].transform(
        lambda x: x.fillna(x.median())
    )
```

Prevención de data leakage en el escalado

El StandardScaler aprende la media y desviación estándar exclusivamente sobre el conjunto de train. Si el scaler aprendiera con test, el modelo tendría acceso indirecto a información futura, invalidando la evaluación:

```
X_train, scaler = scale(X_train, num_cols)
X_test, _ = scale(X_test, num_cols, scaler) # Usa el scaler de train
```

Validación cruzada antes del entrenamiento final

Se realiza validación cruzada de 5 folds para estimar el rendimiento real antes de evaluar sobre test. El modelo se entrena finalmente con todos los datos de train:

```
cv_scores = cross_val_score(model, X_train, y_train, cv=5,
                             scoring='accuracy')
model.fit(X_train, y_train) # Entrenamiento final con todos los datos
```

7. Seguridad implementada

Al tratarse de un proyecto de análisis de datos local sin exposición a red, no se implementan medidas de seguridad en sentido tradicional. Sin embargo, se aplicaron las siguientes buenas prácticas:

- Sin credenciales en el código: ningún archivo contiene claves, tokens o datos sensibles
- Datos anonimizados: el dataset no contiene nombres ni identificadores personales de pacientes
- Reproducibilidad controlada: uso de `random_state=42` en todos los procesos aleatorios
- Modelos serializados localmente: los archivos .pkl se guardan en local y no se exponen externamente

8. Tests y resultados

Validación cruzada (sobre train)

Modelo	CV Accuracy	Desviación
Logistic Regression	84.9%	± 2.6%
Random Forest	84.6%	± 1.9%
XGBoost	84.9%	± 1.0%

Métricas sobre test (184 pacientes)

Modelo	Accuracy	Precision	Recall	F1	AUC
Logistic Regression	89.7%	89.5%	92.2%	90.8%	0.935
Random Forest ✓	90.8%	89.7%	94.1%	91.9%	0.933
XGBoost	88.6%	90.1%	89.2%	89.7%	0.944

Rendimiento por sexo - Recall

Modelo	Hombres	Mujeres
Logistic Regression	93.8%	66.7%
Random Forest ✓	94.8%	83.3%
XGBoost	90.6%	66.7%

Errores críticos del modelo seleccionado (Random Forest)

- Falsos negativos: 6 enfermos no detectados sobre 102 (5.9%)
- Falsos positivos: 11 sanos clasificados como enfermos sobre 82 (13.4%)

9. Decisiones de diseño justificadas

Imputación en lugar de eliminación de filas con ceros

Eliminar las filas con Cholesterol=0 supondría perder una parte significativa del dataset. La imputación por mediana agrupada preserva el tamaño muestral y respeta las diferencias clínicas entre enfermos y sanos.

One-Hot Encoding en lugar de Label Encoding

Las variables categóricas del dataset no tienen orden natural (ASY no es mayor que NAP). Label Encoding implicaría un orden numérico artificial que podría introducir relaciones falsas en el modelo.

StandardScaler en lugar de MinMaxScaler

El dataset contiene outliers en Cholesterol (450-600) y Oldpeak (-2.6 a 6.2). MinMaxScaler es muy sensible a extremos y comprimiría el resto de valores hacia un rango estrecho. StandardScaler es más robusto en presencia de outliers.

Priorizar Recall sobre Accuracy en la selección del modelo

En un clasificador médico, un falso negativo (enfermo no detectado) es mucho más grave que un falso positivo (falsa alarma). Por ello se seleccionó Random Forest por su Recall de 94.1%, aunque XGBoost tiene un AUC ligeramente superior.

Arquitectura modular en src/

Separar el código en preprocessing.py, train_model.py y evaluation.py permite modificar cualquier fase del pipeline sin afectar a las demás, facilita la reutilización y mejora la legibilidad y mantenibilidad del código.

10. Posibles mejoras futuras

- Aumentar la representación de mujeres en el dataset para mejorar el rendimiento del modelo en este subgrupo (actualmente 83.3% de Recall frente a 94.8% en hombres)
- Implementar técnicas de resampling (SMOTE) para corregir el desbalance por sexo sin necesidad de recopilar más datos
- Optimizar hiperparámetros con GridSearchCV o RandomizedSearchCV para mejorar el rendimiento de cada modelo
- Añadir interpretabilidad con SHAP values para explicar las predicciones individuales — especialmente relevante en contexto médico
- Desarrollar una API REST con FastAPI que permita realizar predicciones en tiempo real a partir de datos de nuevos pacientes
- Implementar un pipeline completo con sklearn.Pipeline para encapsular preprocesamiento y modelo en un único objeto serializable